

تقرير البنية التحتية وآلية تحديث البيانات

Infrastructure & Automated Data Update System Report

الإصدار: 1.0

التاريخ: يوليو 2026

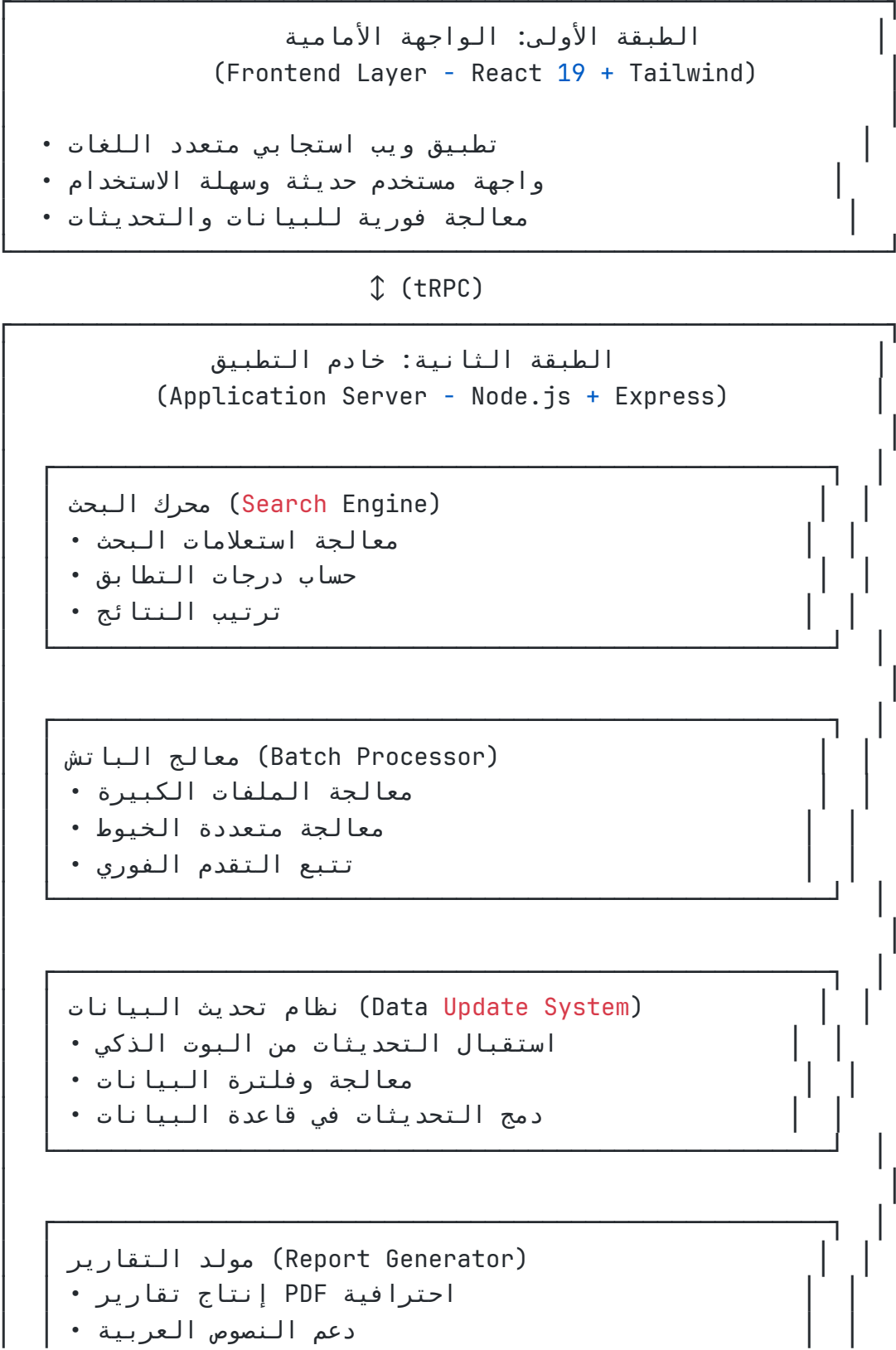
السرية: معلومات حساسة

المحتويات

- البنية التحتية للنظام
- الأساليب التقنية
- الأدوات والمكتبات
- نماذج من الأكواد
- نظام تحديث البيانات الذكي

البنية التحتية للنظام

1.1 معمارية النظام



توقيع رقمي وتشفير •

↕ (SQL)

الطبقة الثالثة: قاعدة البيانات
(Database Layer - MySQL/TiDB + Redis)

- تخزين آمن للبيانات الحساسة
- فهرس متقدمة لتسريع البحث
- نسخ احتياطية دورية
- توافرية عالية (99.9%)

↕

الطبقة الرابعة: نظام تحديث البيانات الذكي
(Intelligent Data Update System - AI Bot)

- (Intelligent Bot) البوت الذكي
- فحص المواقع الرسمية بشكل دوري
 - استخراج البيانات تلقائياً
 - معالجة وتنظيف البيانات
 - إرسال التحديثات إلى النظام

الأساليب التقنية

البحث والمطابقة 2.1

(Fuzzy Search) الأسلوب الأول: البحث الضبابي

"المدخل: محمد علي حسن"



[تطبيع البيانات]



[Fuse.js البحث الضبابي باستخدام]



[Levenshtein Distance حساب درجة التشابه باستخدام]



[ترتيب النتائج حسب الدرجة]



النتائج المرتبة

الأسلوب الثاني: مطابقة الكلمات

الاسم المبحوث عنه: "محمد علي حسن" → [محمد, علي, حسن]



البحث عن كل كلمة في السجل



حساب عدد الكلمات المتطابقة



إذا $3 \leq$ كلمات → تطابق محتمل

إذا $3 >$ كلمات → عدم تطابق

الأسلوب الثالث: معالجة النصوص العربية

"النص العربي: محمد علي حسن"



[إزالة الحروف الخاصة والرموز]



[توحيد الأحرف (تشديد، تنوين)]



[إزالة كلمات التوقف (ال، ب، ل)]



[تطبيع النص النهائي]



"محمد علي حسن"

2.2 معالجة الباتش

(Parallel Processing) الأسلوب: المعالجة المتوازية



الأدوات والمكتبات

3.1 الواجهة الأمامية

الأداة	الإصدار	الاستخدام	الوصف
React	19	بناء الواجهة	لبناء واجهات المستخدم التفاعلية JavaScript مكتبة
Tailwind CSS	4	التصميم	حديث للتصميم السريع CSS إطار عمل
TypeScript	5	البرمجة	لغة برمجة آمنة مع نظام أنواع قوي
Vite	5	البناء	أداة بناء سريعة وحديثة
tRPC	11	الاتصال	مكتبة للاتصال الآمن مع الخادم

3.2 خادم التطبيق

الوصف	الاستخدام	الإصدار	الأداة
على الخادم JavaScript بيئة تشغيل	البيئة	22	Node.js
إطار عمل لبناء تطبيقات الويب	الإطار	4	Express.js
لغة برمجة آمنة مع نظام أنواع قوي	البرمجة	5	TypeScript
أداة للتعامل مع قاعدة البيانات بأمان	قاعدة البيانات	0.30	Drizzle ORM

3.3 معالجة البيانات

الوصف	الاستخدام	الإصدار	الأداة
مكتبة للبحث الضبابي عن النصوص	البحث الضبابي	7	Fuse.js
خوارزمية لحساب درجة التشابه بين النصوص	حساب التشابه	1	Levenshtein
Excel مكتبة لقراءة وكتابة ملفات	Excel قراءة	4	ExcelJS
احترافية PDF مكتبة لإنشاء ملفات	PDF إنشاء	0.13	PDFKit

3.4 الأمان

الوصف	الاستخدام	الإصدار	الأداة
خوارزمية آمنة لتشفير كلمات المرور	تشفير كلمات المرور	5	bcrypt
JWT إنشاء وتحقق من رموز	التحقق من الهوية	9	jsonwebtoken
Node.js مكتبة التشفير المدمجة في	التشفير	Built-in	crypto

قاعدة البيانات 3.5

الوصف	الاستخدام	الإصدار	الأداة
نظام إدارة قواعد البيانات العلائقية	قاعدة البيانات	8	MySQL
نظام قاعدة بيانات موزع وقابل للتوسع	التوسع	Latest	TiDB
نظام تخزين مؤقت سريع في الذاكرة	التخزين المؤقت	7	Redis

نماذج من الأكواد

محرك البحث 4.1

```
// ملف: server/search-engine.ts

import Fuse from 'fuse.js';
import { levenshteinDistance } from 'levenshtein';

interface SearchResult {
  recordId: number;
  name: string;
  matchScore: number;
  matchedWords: number;
  status: 'MATCH' | 'POSSIBLE_MATCH' | 'NO_MATCH';
}

export async function searchSanctionsList(
  query: string,
  language: 'en' | 'ar'
): Promise<SearchResult[]> {
  // تطبيع البيانات
  const normalizedQuery = normalizeText(query);
  const queryWords = normalizedQuery.split(/\s+/).filter(w => w.length > 0);

  // جلب السجلات من قاعدة البيانات
  const records = await db.query(
    `SELECT * FROM sanctionsRecords WHERE nameEn LIKE ? OR nameAr LIKE ?`
    [`${'%${normalizedQuery}%'}`, `${'%${normalizedQuery}%'}`]
  );

  // البحث الضبابي
  const fuse = new Fuse(records, {
    keys: language === 'en' ? ['nameEn'] : ['nameAr'],
    threshold: 0.3,
    minMatchCharLength: 3
  });

  const fuzzyResults = fuse.search(normalizedQuery);

  // حساب درجة التطابق
  const results: SearchResult[] = fuzzyResults.map(result => {
```



```

const record = result.item;
const targetName = language === 'en' ? record.nameEn : record.nameAr;

// حساب درجة التشابه
const similarity = calculateSimilarity(normalizedQuery, targetName);

// حساب عدد الكلمات المتطابقة
const matchedWords = countMatchedWords(queryWords, targetName);

// حساب الدرجة النهائية
const finalScore = (similarity * 0.6) + (matchedWords / queryWords.length);

// تصنيف النتيجة
let status: 'MATCH' | 'POSSIBLE_MATCH' | 'NO_MATCH';
if (finalScore ≥ 0.85 && matchedWords ≥ 3) {
    status = 'MATCH';
} else if (finalScore ≥ 0.60 && matchedWords ≥ 3) {
    status = 'POSSIBLE_MATCH';
} else {
    status = 'NO_MATCH';
}

return {
    recordId: record.id,
    name: targetName,
    matchScore: Math.round(finalScore * 100),
    matchedWords,
    status
};
});

return results.sort((a, b) ⇒ b.matchScore - a.matchScore);
}

function normalizeText(text: string): string {
    return text
        .toLowerCase()
        .replace(/[^\w\s\u0600-\u06FF]/g, '') // إزالة الرموز الخاصة
        .replace(/\s+/g, ' ')
        .trim();
}

function calculateSimilarity(str1: string, str2: string): number {
    const distance = levenshteinDistance(str1, str2);

```

```
    const maxLength = Math.max(str1.length, str2.length);
    return 1 - (distance / maxLength);
}

function countMatchedWords(queryWords: string[], targetName: string): number {
    const targetWords = targetName.split(/\s+/);
    let count = 0;

    for (const queryWord of queryWords) {
        for (const targetWord of targetWords) {
            const similarity = calculateSimilarity(queryWord, targetWord);
            if (similarity ≥ 0.8) {
                count++;
                break;
            }
        }
    }

    return count;
}
```

4.2 معالج الباتش

```
// ملف: server/batch-processor.ts

import { Worker } from 'worker_threads';
import * as ExcelJS from 'exceljs';

interface BatchJob {
  id: string;
  fileName: string;
  totalRecords: number;
  processedRecords: number;
  status: 'PENDING' | 'PROCESSING' | 'COMPLETED' | 'FAILED';
}

export async function processBatchFile(
  filePath: string,
  userId: string
): Promise<BatchJob> {
  // قراءة ملف Excel
  const workbook = new ExcelJS.Workbook();
  await workbook.xlsx.readFile(filePath);
  const worksheet = workbook.getWorksheet(1);

  const records: string[] = [];
  worksheet?.eachRow((row, rowNumber) => {
    if (rowNumber > 1) { // تخطي الرأس
      const name = row.getCell(1).value as string;
      if (name) records.push(name);
    }
  });

  // إنشاء مهمة جديدة
  const job: BatchJob = {
    id: generateId(),
    fileName: filePath.split('/').pop() || 'unknown',
    totalRecords: records.length,
    processedRecords: 0,
    status: 'PROCESSING'
  };

  // حفظ المهمة في قاعدة البيانات
  await db.query(
```

```

    'INSERT INTO screeningJobs (id, userId, fileName, status, totalRecords)
    [job.id, userId, job.fileName, job.status, job.totalRecords]
  );

  // معالجة متعددة الخيوط
  const THREAD_COUNT = 4;
  const BATCH_SIZE = Math.ceil(records.length / THREAD_COUNT);

  const workers = [];
  for (let i = 0; i < THREAD_COUNT; i++) {
    const start = i * BATCH_SIZE;
    const end = Math.min(start + BATCH_SIZE, records.length);
    const batchRecords = records.slice(start, end);

    const worker = new Worker('./batch-worker.ts', {
      workerData: { records: batchRecords, jobId: job.id }
    });

    workers.push(
      new Promise((resolve, reject) => {
        worker.on('message', (result) => {
          job.processedRecords += result.processed;
          resolve(result);
        });
        worker.on('error', reject);
        worker.on('exit', (code) => {
          if (code !== 0) reject(new Error(`Worker exited with code ${code}`));
        });
      })
    );
  }

  // انتظار انتهاء جميع ال workers
  await Promise.all(workers);

  // تحديث حالة المهمة
  job.status = 'COMPLETED';
  await db.query(
    'UPDATE screeningJobs SET status = ?, completedAt = NOW() WHERE id = ?'
    [job.status, job.id]
  );

  return job;
}

```

```
function generateId(): string {  
  return `batch_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`  
}
```

نظام تحديث البيانات 4.3

```
// ملف: server/data-update-system.ts

import axios from 'axios';
import { db } from './db';

interface DataUpdatePayload {
  source: string;
  records: SanctionRecord[];
  timestamp: Date;
}

interface SanctionRecord {
  nameEn: string;
  nameAr?: string;
  entityType: string;
  nationality?: string;
  listingReason: string;
  actionTaken?: string;
}

export async function receiveDataUpdate(
  payload: DataUpdatePayload
): Promise<{ success: boolean; recordsAdded: number; recordsUpdated: number }> {
  let recordsAdded = 0;
  let recordsUpdated = 0;

  for (const record of payload.records) {
    // التحقق من وجود السجل
    const existing = await db.query(
      'SELECT id FROM sanctionsRecords WHERE nameEn = ? AND source = ?',
      [record.nameEn, payload.source]
    );

    if (existing.length > 0) {
      // تحديث السجل الموجود
      await db.query(
        `UPDATE sanctionsRecords
         SET nameAr = ?, entityType = ?, nationality = ?,
           listingReason = ?, actionTaken = ?, updatedAt = NOW()
         WHERE id = ?`,
        [
          record.nameAr, record.entityType, record.nationality,
          record.listingReason, record.actionTaken, record.id
        ]
      );
      recordsUpdated++;
    } else {
      // إضافة سجل جديد
      await db.query(
        'INSERT INTO sanctionsRecords (nameEn, nameAr, entityType, nationality, listingReason, actionTaken, source, timestamp) VALUES (?, ?, ?, ?, ?, ?, ?, ?)',
        [
          record.nameEn, record.nameAr, record.entityType, record.nationality,
          record.listingReason, record.actionTaken, payload.source, payload.timestamp
        ]
      );
      recordsAdded++;
    }
  }

  return { success: true, recordsAdded, recordsUpdated };
}
```

```

        [
            record.nameAr,
            record.entityType,
            record.nationality,
            record.listingReason,
            record.actionTaken,
            existing[0].id
        ]
    );
    recordsUpdated++;
} else {
    // إضافة سجل جديد
    await db.query(
        `INSERT INTO sanctionsRecords
        (nameEn, nameAr, entityType, nationality, listingReason, actionTaken,
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW())`,
        [
            record.nameEn,
            record.nameAr,
            record.entityType,
            record.nationality,
            record.listingReason,
            record.actionTaken,
            payload.source
        ]
    );
    recordsAdded++;
}
}

// تسجيل التحديث
await logDataUpdate({
    source: payload.source,
    recordsAdded,
    recordsUpdated,
    timestamp: payload.timestamp
});

return { success: true, recordsAdded, recordsUpdated };
}

async function logDataUpdate(log: any): Promise<void> {
    await db.query(
        `INSERT INTO dataUpdateLogs

```

```
    (source, recordsAdded, recordsUpdated, timestamp)
    VALUES (?, ?, ?, ?)`,
    [log.source, log.recordsAdded, log.recordsUpdated, log.timestamp]
);
}
```


4.4 مولد التقارير

```
// ملف: server/report-generator.ts

import PDFDocument from 'pdfkit';
import { createWriteStream } from 'fs';

export async function generateScreeningReport(
  jobId: string,
  outputPath: string
): Promise<void> {

  // جلب بيانات المهمة والنتائج
  const job = await db.query(
    'SELECT * FROM screeningJobs WHERE id = ?',
    [jobId]
  );

  const results = await db.query(
    `SELECT sr.*, sr2.nameEn, sr2.nameAr, sr2.nationality, sr2.listingReason
    FROM screeningResults sr
    LEFT JOIN sanctionsRecords sr2 ON sr.matchedRecordId = sr2.id
    WHERE sr.jobId = ?`,
    [jobId]
  );

  // إنشاء ملف PDF
  const doc = new PDFDocument();
  const stream = createWriteStream(outputPath);
  doc.pipe(stream);

  // رأس التقرير
  doc.fontSize(20).text('تقرير فحص العقوبات', { align: 'center' });
  doc.fontSize(12).text(`تاريخ الفحص: ${new Date().toLocaleDateString('ar-EG')}`, { align: 'center' });

  // ملخص النتائج
  doc.moveDown();
  doc.fontSize(14).text('ملخص النتائج', { underline: true });
  doc.fontSize(11).text(`إجمالي الأسماء المفحوصة: ${job[0].totalRecords}`);
  doc.text(`المطابقات الكاملة: ${job[0].matchCount}`);
  doc.text(`المطابقات المحتملة: ${job[0].possibleMatchCount}`);
}
```

```

doc.text(`الأسماء غير المطابقة: ${job[0].noMatchCount}`);

// جدول النتائج
doc.moveDown();
doc.fontSize(14).text('تفاصيل النتائج', { underline: true });

for (const result of results) {
  doc.fontSize(10);
  doc.text(`المفحوص الاسم: ${result.inputName}`);
  doc.text(`الحالة: ${result.status}`);

  if (result.matchedRecordId) {
    doc.text(`الاسم المطابق: ${result.nameEn}`);
    doc.text(`درجة التطابق: ${result.matchScore}%`);
    doc.text(`الجنسية: ${result.nationality}`);
    doc.text(`سبب الإدراج: ${result.listingReason}`);
  }

  doc.moveTo(50, doc.y).lineTo(550, doc.y).stroke();
  doc.moveDown();
}

// تذييل التقرير
doc.moveDown();
doc.fontSize(9).text('تم إنشاء هذا التقرير بواسطة نظام فحص العقوبات', {
  align: 'center'
});

doc.end();

return new Promise((resolve, reject) => {
  stream.on('finish', resolve);
  stream.on('error', reject);
});
}

```

نظام تحديث البيانات الذكي

5.1 البوت الذكي (Intelligent Bot)

البوت الذكي هو العقل المدبر للنظام، وهو مسؤول عن الحفاظ على تحديث قوائم العقوبات بشكل مستمر.

5.1.1 آلية عمل البوت

البوت الذكي

الوظيفة الأساسية:

- فحص المواقع الرسمية بشكل دوري
- استخراج البيانات تلقائياً
- معالجة وتنظيف البيانات
- فلترة البيانات حسب الأعمدة المطلوبة
- إرسال التحديثات إلى النظام

دورة عمل البوت:

(UTC) الساعة 02:00 صباحاً

↓

[فحص موقع الأمم المتحدة]

↓

[استخراج قائمة العقوبات]

↓

[معالجة البيانات]

↓

[فلترة الأعمدة المطلوبة]

↓

[إرسال التحديثات إلى النظام]

↓

[دمج البيانات في قاعدة البيانات]

↓

[تسجيل التحديث]

↓

(UTC) الساعة 02:30 صباحاً

5.1.2 جدول التحديثات الدوري

الحالة	المدة	الوقت	التكرار	المصدر
تلقائي	دقيقة 30	02:00 UTC	يومي	UNSC (الأمم المتحدة)
تلقائي	ساعة 1	03:00 الاثنين UTC	أسبوعي	EU (الاتحاد الأوروبي)
تلقائي	ساعة 1	03:00 الأربعاء UTC	أسبوعي	OFAC (الولايات المتحدة)
تلقائي	ساعة 2	04:00 أول يوم UTC	شهري	القوائم المحلية

5.1.3 خصائص البوت

الخصائص التقنية:

- ✓ معالجة تلقائية بدون تدخل بشري
- ✓ فحص المواقع الرسمية بشكل آمن
- ✓ معالجة البيانات بكفاءة عالية
- ✓ فلترة ذكية للبيانات
- ✓ تقارير مفصلة عن كل تحديث

الموثوقية:

- ✓ إعادة محاولة تلقائية في حالة الفشل
- ✓ معالجة الأخطاء الشاملة
- ✓ تسجيل شامل لجميع العمليات
- ✓ إشعارات فورية عند المشاكل

آلية استقبال التحديثات 5.2

```
// ملف: server/bot-integration.ts

import express from 'express';
import { receiveDataUpdate } from '../data-update-system';

const router = express.Router();

// نقطة نهاية لاستقبال التحديثات من البوت
router.post('/api/bot/update', async (req, res) => {
  try {
    const payload = req.body;

    // التحقق من التوقيع الرقمي
    if (!verifyBotSignature(payload)) {
      return res.status(401).json({ error: 'Invalid signature' });
    }

    // استقبال التحديثات
    const result = await receiveDataUpdate(payload);

    // إرسال الرد
    res.json({
      success: true,
      message: 'Data update received successfully',
      recordsAdded: result.recordsAdded,
      recordsUpdated: result.recordsUpdated
    });

    // إرسال إشعار
    await notifyAdmins({
      type: 'DATA_UPDATE',
      source: payload.source,
      recordsAdded: result.recordsAdded,
      recordsUpdated: result.recordsUpdated,
      timestamp: new Date()
    });

  } catch (error) {
    console.error('Error processing bot update:', error);
    res.status(500).json({ error: 'Failed to process update' });
  }
}
```

```
});
```

```
function verifyBotSignature(payload: any): boolean {  
  // التحقق من التوقيع الرقمي للبوت  
  const signature = payload.signature;  
  const data = JSON.stringify(payload.data);  
  const expectedSignature = crypto  
    .createHmac('sha256', process.env.BOT_SECRET_KEY)  
    .update(data)  
    .digest('hex');  
  
  return signature === expectedSignature;  
}
```

```
async function notifyAdmins(notification: any): Promise<void> {  
  // إرسال إشعار للمسؤولين  
  const admins = await db.query('SELECT email FROM users WHERE role = ?',  
  
  for (const admin of admins) {  
    await sendEmail({  
      to: admin.email,  
      subject: 'تحديث البيانات - نظام فحص العقوبات',  
      body: `  
        تم استقبال تحديث جديد من المصدر  
        السجلات المضافة: ${notification.recordsAdded}  
        السجلات المحدثة: ${notification.recordsUpdated}  
        الوقت: ${notification.timestamp}  
      `,  
    });  
  }  
}
```

```
export default router;
```

5.3 معالجة وفلترة البيانات

```
// ملف: server/data-processing.ts

interface RawData {
  [key: string]: any;
}

interface ProcessedRecord {
  nameEn: string;
  nameAr?: string;
  entityType: string;
  nationality?: string;
  listingReason: string;
  actionTaken?: string;
}

export function processAndFilterData(
  rawData: RawData[],
  requiredColumns: string[]
): ProcessedRecord[] {

  const processedRecords: ProcessedRecord[] = [];

  for (const row of rawData) {
    try {
      // فلترة الأعمدة المطلوبة
      const filteredRow = filterColumns(row, requiredColumns);

      if (!filteredRow.nameEn) {
        console.warn('Skipping row: missing nameEn');
        continue;
      }

      // معالجة البيانات
      const processed: ProcessedRecord = {
        nameEn: normalizeText(filteredRow.nameEn),
        nameAr: filteredRow.nameAr ? normalizeText(filteredRow.nameAr) : null,
        entityType: normalizeEntityType(filteredRow.entityType),
        nationality: filteredRow.nationality,
        listingReason: filteredRow.listingReason || 'Not specified',
        actionTaken: filteredRow.actionTaken
      };
    }
  }
}
```

```

        // التحقق من صحة البيانات
        if (validateRecord(processed)) {
            processedRecords.push(processed);
        }

    } catch (error) {
        console.error('Error processing row:', error);
        continue;
    }
}

return processedRecords;
}

function filterColumns(row: RawData, requiredColumns: string[]): RawData {
    const filtered: RawData = {};

    for (const column of requiredColumns) {
        if (column in row) {
            filtered[column] = row[column];
        }
    }

    return filtered;
}

function normalizeText(text: string): string {
    return text
        .trim()
        .toLowerCase()
        .replace(/\s+/g, ' ');
}

function normalizeEntityType(type: string): string {
    const typeMap: { [key: string]: string } = {
        'individual': 'Individual',
        'person': 'Individual',
        'organisation': 'Organisation',
        'organization': 'Organisation',
        'company': 'Organisation',
        'vessel': 'Vessel',
        'ship': 'Vessel'
    };
};

```



```
    return typeMap[type.toLowerCase()] || 'Unspecified';
}

function validateRecord(record: ProcessedRecord): boolean {
    // التحقق من أن الاسم ليس فارغاً
    if (!record.nameEn || record.nameEn.length < 2) {
        return false;
    }

    // التحقق من نوع الكيان
    const validTypes = ['Individual', 'Organisation', 'Vessel', 'Unspecified'];
    if (!validTypes.includes(record.entityType)) {
        return false;
    }

    return true;
}
```

دمج التحديثات في النظام 5.4

```
// ملف: server/data-integration.ts

export async function integrateUpdates(
  source: string,
  records: ProcessedRecord[]
): Promise<IntegrationResult> {

  const result: IntegrationResult = {
    source,
    totalRecords: records.length,
    addedRecords: 0,
    updatedRecords: 0,
    skippedRecords: 0,
    errors: []
  };

  // بدء معاملة (Transaction)
  const connection = await db.getConnection();
  await connection.beginTransaction();

  try {
    for (const record of records) {
      try {
        // البحث عن السجل الموجود
        const existing = await connection.query(
          'SELECT id FROM sanctionsRecords WHERE nameEn = ? AND source = '
          [record.nameEn, source]
        );

        if (existing.length > 0) {
          // تحديث السجل
          await connection.query(
            `UPDATE sanctionsRecords
              SET nameAr = ?, entityType = ?, nationality = ?,
                  listingReason = ?, actionTaken = ?, updatedAt = NOW()
              WHERE id = ?`,
            [
              record.nameAr,
              record.entityType,
              record.nationality,
              record.listingReason,
```

```

        record.actionTaken,
        existing[0].id
    ]
    );
    result.updatedRecords++;
} else {
    // إضافة سجل جديد
    await connection.query(
        `INSERT INTO sanctionsRecords
        (nameEn, nameAr, entityType, nationality, listingReason, act:
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW())`,
        [
            record.nameEn,
            record.nameAr,
            record.entityType,
            record.nationality,
            record.listingReason,
            record.actionTaken,
            source
        ]
    );
    result.addedRecords++;
}

} catch (error) {
    result.errors.push({
        record: record.nameEn,
        error: (error as Error).message
    });
    result.skippedRecords++;
}
}

// تأكيد المعاملة
await connection.commit();

// تسجيل التحديث
await logIntegration(result);

} catch (error) {
    // إلغاء المعاملة
    await connection.rollback();
    throw error;
}

```

```

    } finally {
      connection.release();
    }

    return result;
  }

  async function logIntegration(result: IntegrationResult): Promise<void> {
    await db.query(
      `INSERT INTO integrationLogs
      (source, totalRecords, addedRecords, updatedRecords, skippedRecords,
      VALUES (?, ?, ?, ?, ?, NOW()))`,
      [
        result.source,
        result.totalRecords,
        result.addedRecords,
        result.updatedRecords,
        result.skippedRecords
      ]
    );
  }

  interface IntegrationResult {
    source: string;
    totalRecords: number;
    addedRecords: number;
    updatedRecords: number;
    skippedRecords: number;
    errors: Array<{ record: string; error: string }>;
  }

```

الخلاصة

البنية التحتية

النظام مبني على بنية حديثة وموثوقة تتكون من

- React و Tailwind واجهة أمامية حديثة باستخدام
- Express و Node.js خادم تطبيق قوي باستخدام
- MySQL/TiDB قاعدة بيانات موثوقة باستخدام

- Redis نظام تخزين مؤقت سريع باستخدام

نظام تحديث البيانات

:البوت الذكي هو العقل المدبر للنظام، وهو يقوم بـ

- ☒ فحص المواقع الرسمية بشكل دوري
- ☒ استخراج البيانات تلقائياً
- ☒ معالجة وتنظيف البيانات
- ☒ فلترة البيانات حسب الأعمدة المطلوبة
- ☒ إرسال التحديثات إلى النظام
- ☒ دمج التحديثات في قاعدة البيانات

.هذا النظام يضمن أن قوائم العقوبات محدثة دائماً وموثوقة

تم إعداد هذا التقرير بواسطة فريق التطوير التقني

الإصدار: 1.0

التاريخ: يوليو 2026

السرية: معلومات حساسة